

**AI Smart Learning Companion:**  
**An AI-Powered Study Workflow Assistant for Graduate Students**

Harshitha Chennareddy, Hotragn Pettugani, Mengxia Qiu

Group 10

Northeastern University

INFO 7375 Prompt Engineering for Generative AI

Professor: Shirali Patel

February 22, 2026

# **AI Smart Learning Companion:**

## **An AI-Powered Study Workflow Assistant for Graduate Students**

### **1. App Concept & Problem Statement**

As part of our "AI Smart Learning Companion", we focused on the **Quiz Generation Module**, driven by our system's **“Socratic Tutor Persona”**. Graduate students routinely manage dense lecture materials. Educational psychology research indicates that passive reading often leads to a false sense of fluency, whereas active retrieval practice significantly improves long-term retention.

Our module transforms static course content into an interactive learning loop by generating structured, graduate-level diagnostic quizzes. The critical Prompt Engineering challenge is maintaining strict output formatting for backend integration while ensuring the AI grounds its questions strictly in the provided notes (Context-Grounded Generation) without hallucination.

***Note on Documentation Length:** To comply with the report length constraints, we have included screenshots for key foundational and advanced milestones (Zero-shot, Few-shot, CoT, Decomposition/Ensembling/Self-Consistency, and Self-Criticism). All other mentioned techniques were successfully executed and rigorously evaluated in our environment; their empirical observations are summarized below without screenshots for brevity.*

### **2. Initial System Prompt: Exploring Foundational Structures**

In this phase, we applied foundational prompting techniques to the same task (generating a quiz from fixed lecture notes). By keeping the task and input constant, we directly compared

how each technique shaped clarity, structure, reasoning quality, and reliability to draft a well-reasoned initial system prompt.

## 2.1 Zero-Shot Prompting

To test the model's baseline capabilities without any scaffolding, we provided only the base instructions and the lecture notes(see Figure 1).

**Figure 1**

*Zero-Shot Prompting Code and Output*

```
# =====  
# Technique 1: Zero-shot  
# =====  
user_zero = f"""  
Create a diagnostic quiz based ONLY on the lecture notes below.  
  
Requirements:  
- 3 Multiple-Choice Questions (Easy, Medium, Hard)  
- 2 Short Answer Questions  
- 1 Application Question  
- Include answer key  
- Include 1-2 line explanation per answer  
  
Lecture Notes:  
{LECTURE_NOTES}  
""".strip()  
  
zero_out = run_prompt(BASE_SYSTEM_PROMPT, user_zero)
```

```

===== ZERO-SHOT OUTPUT =====

### Diagnostic Quiz: Prompt Engineering and Model Behavior

#### Multiple-Choice Questions

**Question 1 (Easy):**
What does zero-shot prompting rely on?
A) Task examples
B) Pretrained knowledge
C) Intermediate steps
D) Structured outputs

**Answer:** B) Pretrained knowledge
*Explanation: Zero-shot prompting utilizes the model's existing knowledge without providing specific examples of the task.*

---

**Question 2 (Medium):**
Which prompting technique is specifically designed to improve performance on reasoning-heavy tasks?
A) Stepback prompting
B) Few-shot prompting
C) Chain-of-thought prompting
D) Self-consistency techniques

**Answer:** C) Chain-of-thought prompting
*Explanation: Chain-of-thought prompting encourages models to articulate intermediate steps, enhancing their reasoning capabilities.*

---

```

\*(Output truncated for brevity)

```

#### Application Question

**Question 6:**
Imagine you are tasked with designing a prompt for a complex reasoning task. Describe how you would utilize chain-of-thought and stepback prompting.

**Answer:**
To design a prompt for a complex reasoning task, I would first employ stepback prompting by framing the question in a way that encourages the model to identify high-level principles before diving into detailed reasoning.

---

### Answer Key Summary
1. B
2. C
3. B
4. Few-shot prompting reduces ambiguity by providing examples, enhancing performance.
5. Stepback prompting abstracts high-level principles to improve detailed reasoning.
6. Use stepback prompting to identify principles, then chain-of-thought prompting for structured reasoning.

```

*Note.* The top image displays the zero-shot instruction provided to the model. The subsequent images show the unformatted baseline quiz generated.

**Reflection:** Relying entirely on pretrained knowledge without providing examples, the model produced generally accurate questions, demonstrating it can perform the task without additional scaffolding. However, the formatting and structural consistency varied slightly, highlighting that zero-shot prompting is highly sensitive to instruction phrasing and leads to variability in presentation.

## 2.2 Few-Shot Prompting

To address the formatting variability observed in the zero-shot baseline, we injected a structured sample quiz into the prompt (see Figure 2).

**Figure 2**

*Few-Shot Prompting Code and Output*

```
# =====
# Technique 2: Few-shot
# =====
few_shot_example = """
Example Notes:
Topic: Overfitting in Machine Learning
- Overfitting occurs when a model memorizes noise in training data.
- It leads to high training accuracy but poor generalization.
- Regularization and more training data reduce overfitting.

Example Quiz Format:

1) MCQ (Easy):
Question: What is overfitting?
A) High bias
B) Memorizing training noise
C) Undertraining
D) Data imbalance
Answer: B
Explanation: Overfitting means memorizing training noise instead of learning general patterns.

2) Short Answer:
Question: Name one method to reduce overfitting.
Answer: Regularization.
Explanation: Regularization discourages memorization and improves generalization.
""".strip()

user_few = f"""
{few_shot_example}

Now generate a quiz in the SAME format using the lecture notes below.

Requirements:
- 3 MCQ (Easy/Medium/Hard)
- 2 Short Answer
- 1 Application
Each must include Answer + 1-2 line Explanation.

Lecture Notes:
{LECTURE_NOTES}
""".strip()

few_out = run_prompt(BASE_SYSTEM_PROMPT, user_few)
```

```

===== FEW-SHOT OUTPUT =====

### Quiz on Prompt Engineering and Model Behavior

1) **MCQ (Easy)**:
  **Question:** What is zero-shot prompting?
  A) Providing multiple examples of a task
  B) Relying on pretrained knowledge without task examples
  C) Encouraging intermediate reasoning steps
  D) Abstracting to high-level principles
  **Answer:** B
  **Explanation:** Zero-shot prompting utilizes the model's existing knowledge without providing specific examples of the task.

2) **MCQ (Medium)**:
  **Question:** How does few-shot prompting improve model performance?
  A) By relying solely on pretrained knowledge
  B) By demonstrating expected format and reasoning patterns
  C) By generating multiple outputs for consistency
  D) By abstracting to high-level principles
  **Answer:** B
  **Explanation:** Few-shot prompting reduces ambiguity by providing examples that clarify the expected format and reasoning for the task.

```

\*(Output truncated for brevity)

```

5) **Short Answer**:
  **Question:** Describe the purpose of stepback prompting.
  **Answer:** Stepback prompting helps the model abstract to high-level principles before engaging in detailed reasoning.
  **Explanation:** This technique allows the model to frame the problem conceptually, which can lead to more effective reasoning.

6) **Application**:
  **Question:** How would you apply few-shot prompting to improve a model's performance on a new task?
  **Answer:** I would provide the model with a few examples that illustrate the desired output format and reasoning process for the new task.
  **Explanation:** By demonstrating the expected patterns through examples, few-shot prompting clarifies the task for the model, reducing ambiguity and enhancing its ability to generate accurate responses.

```

*Note.* The top image shows the structured sample quiz injected into the prompt. The subsequent images display the resulting formatted quiz output, demonstrating how a single example improves structural consistency.

**Reflection:** The inclusion of a structured sample quiz improved formatting consistency and clarity. The model adhered closely to the demonstrated structure, producing cleaner labeling of difficulty levels and more uniform explanations. This shows that even a single example can reduce ambiguity and improve reproducibility without adding much implementation complexity.

## 2.3 Chain-of-Thought (CoT) Prompting

To elevate the logical depth of the quiz, we forced the model to explicitly reason through the concepts before generating questions(see Figure 3).

**Figure 3**

## Chain-of-Thought Prompting Code and Output

```
# =====
# Technique 3: Chain-of-thought (structured guided reasoning)
# =====
user_cot = f"""
Follow these steps:

Step 1: Extract 5 key conceptual insights from the lecture notes.
Step 2: Map each insight to a question type (MCQ, Short Answer, Application) and difficulty.
Step 3: Generate the final quiz:
    - 3 MCQ (Easy/Medium/Hard)
    - 2 Short Answer
    - 1 Application
Step 4: Provide an answer key and 1-2 line explanation per question.

Lecture Notes:
{LECTURE_NOTES}
""".strip()

cot_out = run_prompt(BASE_SYSTEM_PROMPT, user_cot)
```

```
===== CHAIN-OF-THOUGHT (STRUCTURED) OUTPUT =====

### Step 1: Key Conceptual Insights

1. Sensitivity to Instruction Phrasing: Model outputs are significantly influenced by how instructions are phrased and structured.
2. Zero-shot Prompting: This technique relies solely on the model's pretrained knowledge without providing specific task examples.
3. Few-shot Prompting: This method mitigates ambiguity by providing examples of the expected format and reasoning patterns.
4. Chain-of-Thought Prompting: This approach enhances performance on reasoning-heavy tasks by promoting the articulation of intermediate steps.
5. Self-Consistency Techniques: These techniques aim to reduce output variance by generating multiple reasoning paths and selecting the most consistent result.

### Step 2: Mapping Insights to Question Types



| Insight                             | Question Type | Difficulty |
|-------------------------------------|---------------|------------|
| Sensitivity to Instruction Phrasing | MCQ           | Medium     |
| Zero-shot Prompting                 | MCQ           | Easy       |
| Few-shot Prompting                  | Short Answer  | Medium     |
| Chain-of-Thought Prompting          | MCQ           | Hard       |
| Self-Consistency Techniques         | Application   | Hard       |



### Step 3: Final Quiz

#### Multiple Choice Questions (MCQ)

1. Easy What does zero-shot prompting rely on?
    - A) Task examples
    - B) Pretrained knowledge
    - C) Intermediate reasoning steps
    - D) High-level principles
```

\*(Output truncated for brevity)

```
#### Application Question

6. Design a prompt for a language model using few-shot prompting to generate a summary of a scientific article. Include at least two examples of expected output.
Answer:
Prompt: "Below are examples of summaries for scientific articles. Based on the following article, please generate a summary.
Example 1: 'The study investigates the effects of climate change on polar bear populations, highlighting the decline in sea ice and its impact on hunting patterns.'
Example 2: 'This research explores the relationship between diet and heart health, demonstrating that a plant-based diet can significantly reduce the risk of cardiovascular diseases.'
Article: [Insert article text here].
Summary: [Your summary here]."
```

```
### Step 4: Answer Key and Explanations

1. Answer: B - Zero-shot prompting relies on the model's pretrained knowledge, as it does not provide specific task examples.
2. Answer: B - The phrasing of instructions can significantly alter the outputs, as models are sensitive to how tasks are presented.
3. Answer: C - Chain-of-thought prompting improves reasoning performance by encouraging the model to articulate intermediate steps, enhancing clarity and accuracy.
4. Answer: Few-shot prompting helps improve model performance by providing examples that clarify the expected output format and reasoning, thus reducing ambiguity.
5. Answer: Self-consistency techniques generate multiple reasoning paths to ensure that the output is consistent, which helps in reducing variance and increasing reliability.
6. Answer: The prompt provides a structured approach to few-shot prompting by including examples that guide the model in generating a summary, showcasing how to apply the concept effectively.
```

*Note.* The top image illustrates the prompt forcing explicit reasoning steps. The subsequent images capture the model's intermediate cognitive process (e.g., concept extraction) prior to generating the actual quiz questions.

**Reflection:** By instructing the model to first extract key concepts and then generate questions, the output became more systematic and aligned with the lecture material. Although this approach produced longer responses, reasoning transparency and logical flow improved noticeably, ensuring better coverage of important ideas compared to the baseline.

## 2.4 Additional Foundational Tests: Stepback & Generate-Knowledge

*(Screenshots omitted for brevity)*

To complete our initial draft, we also tested **Stepback Prompting** and **Generate-Knowledge Prompting**:

- **Stepback Prompting:** This technique encouraged abstraction to high-level principles before engaging in detailed reasoning. It resulted in stronger conceptual framing, helping the model ground the quiz in broader conceptual themes before moving to specific, isolated question construction.
- **Generate-Knowledge Prompting:** Requiring the model to first extract structured knowledge improved grounding and reduced the risk of including loosely connected or redundant questions. The resulting quiz felt more tightly aligned with the lecture notes, forming the robust foundation of our well-reasoned first draft.

## 3. Advanced System Prompt: Robustness & Quality Control



To further improve robustness, reduce variance, and correct weaknesses, we applied advanced prompt engineering strategies, transforming the initial draft into a highly reliable, production-ready system prompt.

### 3.1 Intermediate Advanced Tests (Variance Reduction & Complexity Handling)

Before reaching our final prompt architecture, we rigorously tested several advanced techniques to handle cognitive load and evaluate output variance:

- **Decomposition:** We conceptually separated knowledge extraction from quiz generation. This step successfully reduced the likelihood of missing important concepts, effectively handling the cognitive complexity of generating a multifaceted assessment.
- **Ensembling:** We generated multiple candidate quizzes using the same prompt, which revealed natural variability across runs (e.g., differences in emphasis or phrasing). This highlighted the necessity of an automated selection mechanism.
- **Self-Consistency & Universal Self-Consistency:** To mitigate the variability observed during ensembling, we implemented a mathematical filtering mechanism. By selecting the strongest candidate using defined evaluation criteria (*groundedness, completeness, and clarity*), we drastically reduced output variability and improved overall reliability (see Figure 4).

#### Figure 4

*Decomposition, Ensembling, and Self-Consistency Pipeline*

```

# =====
# ADVANCED SYSTEM PROMPT
# Decomposition + Ensembling + Self-consistency
# =====

ADVANCED_SYSTEM_PROMPT = """
You are the AI Smart Learning Companion operating in Socratic Tutor mode.

Non-negotiable rules:
- Use ONLY the provided lecture notes.
- Do NOT add outside facts or examples.
- Output must be structured and reproducible.

Advanced workflow (Decomposition):
1) Extract the key concepts from the lecture notes as a short concept list.
2) Use ONLY that concept list to generate a diagnostic quiz.

Output requirements:
- 3 MCQ labeled (Easy/Medium/Hard) with A/B/C/D options
- 2 Short Answer
- 1 Application
- Include answer key
- Include 1-2 line explanation per answer
"""

advanced_user = f"""
Lecture Notes:
{LECTURE_NOTES}

Generate the diagnostic quiz using the workflow and output requirements.
"""

# Ensembling: generate multiple candidates
advanced_candidates = ensemble_generate(ADVANCED_SYSTEM_PROMPT, advanced_user, n=3, temperature=0.85)

print("\n===== ADVANCED: ENSEMBLED CANDIDATES =====\n")
for i, c in enumerate(advanced_candidates, start=1):
    print(f"\n--- Candidate {i} ---\n{c}\n")

# Self-consistency: pick best candidate
advanced_judgement = judge_best_candidate(LECTURE_NOTES, advanced_candidates)

print("\n===== ADVANCED: SELF-CONSISTENCY SELECTION =====\n")
print(advanced_judgement)

```

```

===== ADVANCED: ENSEMBLED CANDIDATES =====

--- Candidate 1 ---
### Key Concepts
1. Sensitivity of model outputs to instruction phrasing and structure
2. Zero-shot prompting
3. Few-shot prompting
4. Chain-of-thought prompting
5. Stepback prompting
6. Self-consistency techniques
7. Structured outputs

### Diagnostic Quiz

#### Multiple Choice Questions

**Easy:**
1. What type of prompting relies entirely on pretrained knowledge without task examples?
  A) Few-shot prompting
  B) Chain-of-thought prompting
  C) Zero-shot prompting
  D) Stepback prompting
**Answer:** C
*Explanation: Zero-shot prompting does not use any examples and relies solely on pre-existing knowledge.*

```

\*(Output truncated for brevity)

```

===== ADVANCED: SELF-CONSISTENCY SELECTION =====

- Chosen Candidate: 1
- Why:
  - **Groundedness**: Candidate 1 strictly adheres to the lecture notes without incorporating any outside information.
  - **Completeness**: It includes exactly 3 MCQs (Easy, Medium, Hard), 2 Short Answer questions, and 1 Application question, fulfilling the requirements.
  - **Structure**: The formatting is clean and organized, with clear sections for each type of question and answers accompanied by concise explanations.
  - **Coverage**: It effectively reflects all major points from the notes, including prompt sensitivity, zero-shot, few-shot, chain-of-thought, stepback, self-consistency.
  - **Clarity**: The language is student-friendly, with clear questions and explanations that minimize confusion.

- Chosen Text (verbatim):
### Key Concepts
1. Sensitivity of model outputs to instruction phrasing and structure
2. Zero-shot prompting
3. Few-shot prompting
4. Chain-of-thought prompting
5. Stepback prompting
6. Self-consistency techniques
7. Structured outputs

### Diagnostic Quiz

#### Multiple Choice Questions

**Easy:**
1. What type of prompting relies entirely on pretrained knowledge without task examples?
  A) Few-shot prompting
  B) Chain-of-thought prompting

```

*Note.* The top image displays the advanced workflow combining decomposition, ensembling generation, and a self-consistency judge. The middle image shows one of the generated ensemble candidates. The bottom image demonstrates the self-consistency selection process, evaluating candidates against predefined criteria (groundedness, completeness, structure, coverage, and clarity) to automatically select the optimal output.

### 3.2 The Final Pipeline: Self-Criticism

As a final layer of quality assurance, we commanded the model to autonomously critique and revise its own generated quiz against a strict checklist (see Figure 5).

**Figure 5**

#### *Self-Criticism Revision Checklist and Output*

```
===== SELF-CRITICISM + REVISED FINAL =====

A) Critique
- The draft uses lecture notes exclusively, which is good.
- The quiz includes exactly 3 MCQs with appropriate difficulty levels and options.
- There are exactly 2 Short Answer questions.
- There is 1 Application question.
- The answer key is present, but it incorrectly numbers the answers (it includes answers for questions 4, 5, and 6 which do not exist in the quiz).
- The explanations for the answers are clear and concise.
- The formatting is mostly clear, but the numbering for the answer key is inconsistent with the quiz questions.
- There are no missing answers/options or duplicated numbering.

B) Revised Final Output

### Key Concepts
1. Sensitivity of model outputs to instruction phrasing and structure
2. Zero-shot prompting
3. Few-shot prompting
4. Chain-of-thought prompting
5. Stepback prompting
6. Self-consistency techniques
7. Structured outputs

### Diagnostic Quiz

#### Multiple Choice Questions

**Easy:**
1. What type of prompting relies entirely on pretrained knowledge without task examples?
A) Few-shot prompting
```

\*(Output truncated for brevity)

```
#### Short Answer Questions

**1. Describe the concept of few-shot prompting.**
**Answer:** Few-shot prompting reduces ambiguity by providing examples that demonstrate the expected format and reasoning pattern.

**2. What is stepback prompting, and how does it assist in reasoning tasks?**
**Answer:** Stepback prompting helps by abstracting to high-level principles before engaging in detailed reasoning. This approach allows the model to identify the core concepts and structure of the problem before diving into specific details.

#### Application Question

**3. In a scenario where a model is tasked with generating a report, how could the use of self-consistency techniques improve the output quality and reliability?
**Answer:** By generating multiple reasoning paths using self-consistency techniques, the model can identify the most consistent and reliable information, reducing the risk of hallucinations or errors.

### Answer Key
1. C
2. C
3. C
4. Few-shot prompting reduces ambiguity by providing examples.
5. Stepback prompting abstracts to high-level principles to aid reasoning.
6. Self-consistency techniques improve output reliability.
```

*Note.* The top image displays the self-criticism prompt along with the predefined evaluation checklist. The subsequent images show the model autonomously critiquing and revising its own initial output to correct structural or factual inconsistencies.

**Reflection:** This critical step required the model to act as its own evaluator. By critiquing and revising its output against a defined checklist, it significantly improved structural completeness and corrected minor factual inconsistencies. Combined with the foundational techniques, this self-correcting mechanism resulted in a truly production-ready assessment generation pipeline.

#### **4. Conclusion: The Evolution to a Production-Ready Prompt**

Across all experiments, prompt structure clearly influenced model behavior. The evolution from zero-shot to advanced prompting demonstrates that systematic refinement significantly improves consistency, usability, and robustness.

We identified clear trade-offs: clarity and reproducibility increase as prompts become more explicit, but this also increases prompt length and development effort. **Few-shot prompting** offered the best balance between effort and UI formatting improvement, while **advanced techniques (Self-Consistency, Self-Criticism)** provided the greatest gains in reliability and variance reduction.

Ultimately, rather than relying solely on raw model capability, deliberate prompt engineering plays a central role in shaping reliable AI system behavior. Moving forward, our *AI Smart Learning Companion* will utilize this advanced, chained prompt architecture to deliver highly dependable, hallucination-free educational tools.

## References

- Kim, P., Wang, W., & Bonk, C. J. (2025). Generative AI as a coach to help students enhance proficiency in question formulation. *Journal of Educational Computing Research*, 63(3), 565–586. <https://doi.org/10.1177/07356331251314222>
- Lee, D., & Palmer, E. (2025). Prompt engineering in higher education: A systematic review to help inform curricula. *International Journal of Educational Technology in Higher Education*, 22(7), 1–22. <https://doi.org/10.1186/s41239-025-00503-7>
- Roediger, H. L., & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. *Psychological Science*, 17(3), 249–255. <https://doi.org/10.1111/j.1467-9280.2006.01693.x>
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285. [https://doi.org/10.1207/s15516709cog1202\\_4](https://doi.org/10.1207/s15516709cog1202_4)